



**FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY
AND RESEARCH**

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y. 2026-27 Semester-3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



Practical-3

Practical AIM:

To implement and compare multiple sorting algorithms and develop the ability to reason about which sorting strategy suits a given situation.

3.1

Problem Definition:

A teacher has a stack of student answer sheets with marks written on them and needs to arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front; in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets. Implement all three methods and for each one, print the sorted order of marks.

Describe how each of the three methods works in your own words. If the sheets were already in order before the teacher started, which method would finish the fastest and why?

Output:

```
Enter no of students:3
Enter marks:67 45 93

Bubble sort:31 45 67

Selection sort:45 67 93

Insertion sort:45 67 93
```

Difficulty faced:

The main difficulty was understanding the working of all three sorting methods and implementing their swapping and shifting correctly. It was also important to handle the already sorted case efficiently.

Solution:

- Bubble Sort: Compared adjacent marks and swapped them whenever they were in the wrong order. Repeated this until the list was sorted.
- Selection Sort: Found the smallest mark from the unsorted part and placed it at the correct position.
- Insertion Sort: Took each mark and inserted it into its correct position among the already sorted marks.

Outcome:

Successfully implemented all three sorting techniques and obtained the sorted order of marks. The practical helped in understanding the differences between Bubble Sort, Selection Sort, and Insertion Sort. Among the three, Insertion Sort performs fastest when the data is

already sorted, as it can complete with approximately $O(n)$ operations.

3.2

Problem Definition:

A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result.

Describe the approach you used. Does your solution make more than one pass through the list? Can you think of a way to do it in exactly one pass?

Output:

```
Enter no of elements:4
Enter 0,1 or 2:2
0
0
1
Sorted elements:0 0 1 2
```

Difficulty faced:

The main difficulty was arranging 0, 1, and 2 in the correct order without using another array. Managing the positions while swapping elements was also a little challenging.

Solution:

I used the counting approach. First, I counted how many 0s, 1s, and 2s are present in the array. Then I replaced the elements in the same array according to their counts: first all 0s, then 1s, and finally 2s.

Outcome:

Successfully rearranged the colour codes so that all 0s come first, followed by 1s and then 2s. The solution works without using an extra array and helped in understanding in-place rearrangement. The counting method takes two passes, while the three-pointer method can solve it in one pass.

Key Questions:

1. [Problem 1] One of the three methods can detect early that the list is already sorted and stop without doing any more work. Which method is it, how would you add that detection, and why can the other two not do the same?

Sol: Bubble Sort can detect that the list is already sorted. Add a swapped variable and set it to true whenever a swap occurs. If no swap happens in a complete pass, stop the loop. Selection Sort and Insertion Sort can also be optimized in some ways, but Bubble Sort has this early-stop check most directly.

2. [Problem 2] If your solution makes two passes through the list, can you identify which decision in your logic forces the second pass? Is there a way to restructure that decision to eliminate it?

Sol: The second pass is needed when the solution first counts the number of 0s, 1s, and 2s, and then goes through the array again to place them. To eliminate the second pass, the three-pointer approach can rearrange elements while scanning the array, so no separate rewriting pass is needed.

Github Link:

<https://github.com/brindajivani/DSA/tree/main/Practical3>